

MiniWare documentación

Álvaro González Luque

11 de mayo de 2026

Índice

1. Leyenda	2
2. Personalización de la interfaz	2
3. Recursos de nivel (Carpeta Levels/ y subcarpetas Level(N)/)	4
4. Minijuegos (Carpeta Minigames/ y subcarpetas Minigame(N)/)	7
5. EXTRA: Puntuaciones codificadas	8

MiniWare es un *framework* diseñado para facilitar la creación de videojuegos del estilo *WarioWare*. En él podrás personalizar tu propio videojuego de forma sistemática aprovisionando las carpetas especificadas con los distintos recursos que desees añadir, empleando la nomenclatura correcta, descrita en este documento.

1. Leyenda

Los recursos se agrupan de acuerdo a los siguientes tipos y subtipos. Cada uno de ellos permite introducir los tipos de extensiones que se indican a continuación:

■ Elementos visuales

- **Imagen (I)**: permite imágenes con cualquiera de los siguientes formatos.
bmp, dds, ktx, exr, hdr, jpg, jpeg, png, tga, svg, webp.

- **Vídeo (V)**: permite vídeos sin transparencias únicamente de formato ogv.

Se recomienda no emplear vídeos demasiado grandes o con una longitud mayor que la que se esté reproduciendo, ya que podría incrementar el tamaño del juego de forma excesiva.

- **Shader (S)**: permite archivos de código de formato `gdshader`

Para más información, la comunidad de *Godot* proporciona una gran cantidad de *shaders* de uso libre en la página *Godot Shaders*.

■ Elementos sonoros (M)

Para introducir tanto pistas de audio para la banda sonora del juego como sonidos cortos puntuales. Esto es posible empleando los siguientes formatos: ogg, mp3 o wav.

■ Elementos de texto

- **JSON (J)**: los archivos JSON serán empleados para especificar la información de distintos campos necesarios que serán especificados más adelante.
- **Texto plano (T)**: archivos de formato `txt` que posibilitan el uso de etiquetas *BBCode*.

2. Personalización de la interfaz

A continuación se enumerarán las distintas carpetas existentes para la personalización de la interfaz y los nombres que deben tener cada uno de sus elementos para ser incorporados correctamente. La extensión de estos archivos, habiendo indicado los posibles tipos, tiene que corresponder con alguna de las especificadas en la leyenda. Se indica el formato que adquirirán en las figuras 1 (para la pantalla de título), 2 (para la pantalla de créditos) y 3 (para la pantalla del menú principal).

■ Pantalla de título:



Figura 1: Elementos de la pantalla de título

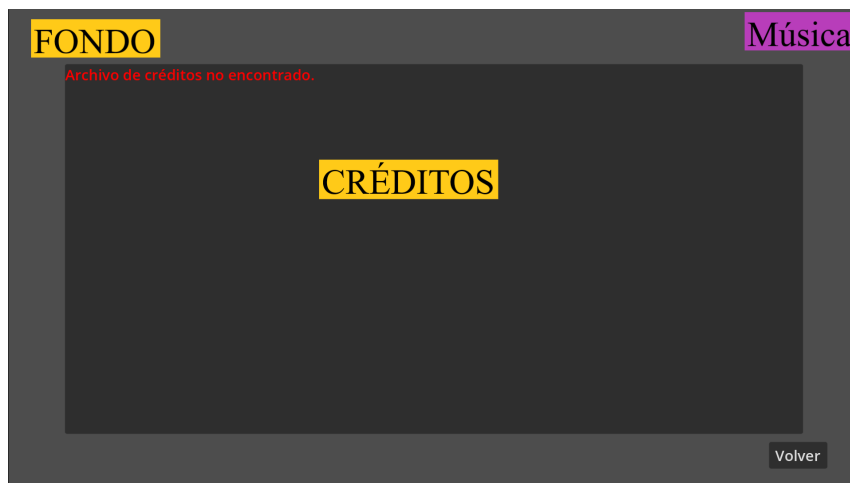


Figura 2: Elementos de la pantalla de créditos

- titleBg: Fondo. (S, V, I)
- logo: Logo. (S, V, I)
- titleTheme: Música. (M)
- **Pantalla de créditos:**
 - creditsBg: Fondo. (S, V, I)
 - credits.txt: Créditos. (T)
 - creditsTheme: Música. (M)
- **Menú principal:**
 - mainMenuBg: Fondo. (S, V, I)
 - mainMenuTheme: Música. (M)
 - icon: Botón de *play*. (S, I)

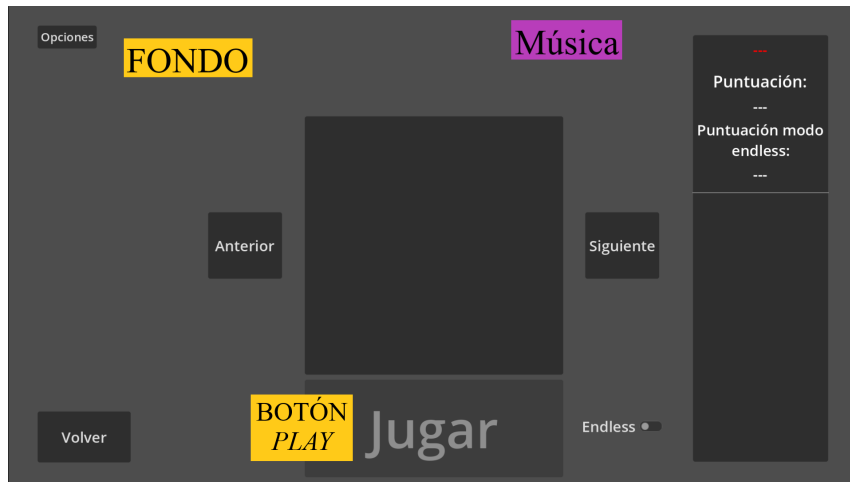


Figura 3: Elementos de la pantalla del menú principal

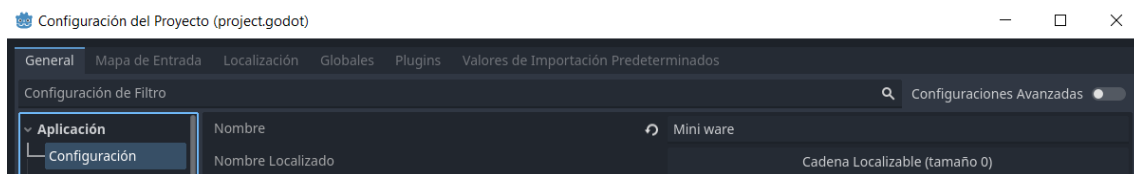


Figura 4: El nombre del proyecto se modifica en la configuración del mismo.

Por último, para mostrar el nombre del juego en la pantalla de título este se obtiene directamente a través del nombre del proyecto. Para modificarlo es necesario acceder a la configuración del proyecto y editar el campo “Nombre” en la configuración general de la aplicación, figura 4.

3. Recursos de nivel (Carpeta Levels/ y subcarpetas Level(N)/)

Por defecto existirá una carpeta **Levels** en el proyecto. En ella, por cada otra carpeta existente, el menú principal ofrecerá la opción de jugar el nivel con los recursos que se hayan administrado. Estos recursos se recogen a continuación y se ejemplifican en las figuras 5, 6, 7.

- **Carpeta Videos/:** contiene los vídeos empleados a modo de animación durante la ejecución del nivel. Se distingue entre las siguientes escenas: **intro** (cinemática introductoria al nivel), **control** (escena principal de control), **loseVid** (escena de control en caso de derrota en el minijuego anterior), **winEndVid** (cinemática final para cuando el nivel ha sido superado) y **loseEndVid** (cinemática final para cuando se pierden todas las vidas). (V)
- **Carpeta Popups/:** contiene las imágenes emergentes que aparecen durante el nivel. Entre ellas encontramos **speedUp**, que se muestra al alcanzar la puntuación especificada para el evento de aumento de la velocidad, y los distintos *popups* de instrucciones, nombrados según indique la información del minijue-

go próximo a ejecutar. (S, V, I)

- **Carpeta Clocks/:** contiene los recursos empleados para representar la advertencia sobre el tiempo restante, a modo de reloj animado. El nombre de los recursos puede ser cualquiera y se reproducirán siguiendo el orden alfabético, en tantos intervalos de tiempo como elementos haya, repartidos en torno a los dos segundos de advertencia. (S, I)
- **healthSprite:** usado para indicar las vidas restantes en la escena de control. (I)
- **info:** Contiene la información específica del nivel. Los campos se indican a continuación. (J)
 - *description:* Contiene un texto con la descripción del nivel, de manera que se muestre en su panel correspondiente en la escena del menú principal.
 - *level_name:* nombre del nivel mostrado en la escena del menú principal.
- **levelTheme:** Música reproducida durante el nivel. Si se incluye, se silencian los vídeos empleados en las escenas de control. (S, V, I)
- **picture:** Imagen sobre el nivel que será mostrada en el menú principal. (S, V, I)
- **speedUpSFX:** Sonido que se reproducirá al lanzar el evento de aumento de la velocidad. (S, V, I)
- **_index.json** de niveles: Es necesario incluir un archivo `res://Public/Levels/_index.json` con un array que contenga el nombre exacto de cada carpeta de nivel:

```
["Level1", "Level2", "Level3"]
```

El orden del array determina el orden en que aparecen los niveles en el menú principal. Los nombres deben coincidir exactamente con los nombres de las carpetas.

- **_index.json** de minijuegos: Dentro de cada nivel, crea el archivo `res://Public/Levels/LevelX/Minigames/_index.json` con un array que contenga el nombre de cada carpeta de minijuego:

```
["Minigame1", "Minigame2", "Minigame3"]
```

De nuevo, los nombres deben coincidir exactamente con los nombres de las carpetas.

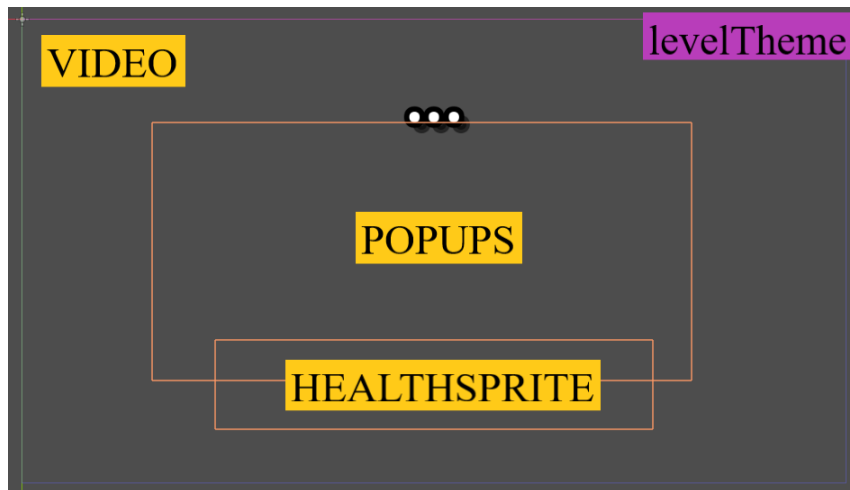


Figura 5: Elementos de la escena de control.

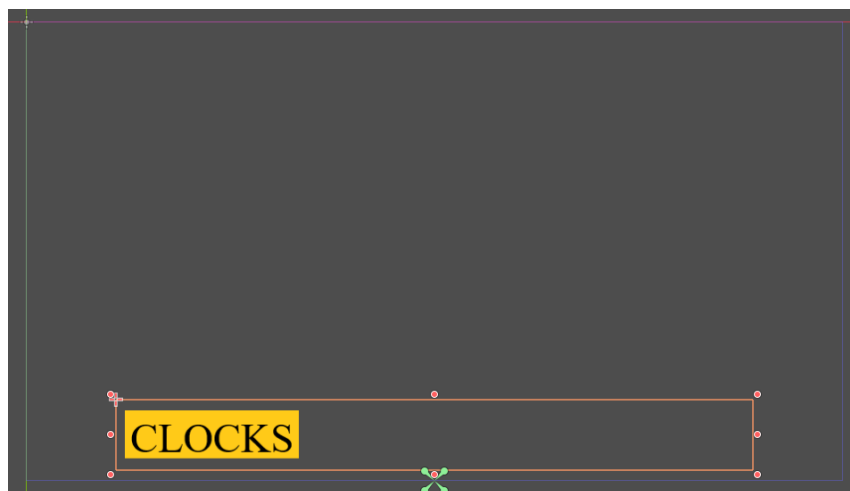


Figura 6: Elementos del minijuego.



Figura 7: Elementos del nivel en el menú principal.

4. Minijuegos (Carpeta Minigames/ y subcarpetas Minigame(N)/)

Cada carpeta de nivel deberá incluir una carpeta `Minigames`, donde se deben añadir (separados en subcarpetas) cada uno de los minijuegos que se espera incluir en el mismo. Para que se reconozca un minijuego, debe existir en su carpeta, al menos, una escena principal con el nombre `Game.tscn`. En caso de que no exista, el botón de *play* en el menú principal será desactivado.

Además, cada minijuego debe incluir un archivo `info.json` (J) con especificaciones sobre el mismo. Los campos a incluir en el mismo se indican a continuación.

- *duration*: duración en segundos del minijuego en la dificultad inicial.
- *instruction*: nombre del recurso en la carpeta `Popups` del nivel con la instrucción adecuada para el minijuego.
- *survival*: booleano que indica si la señal del minijuego es de victoria o derrota al terminar el temporizador.

Para el correcto funcionamiento del minijuego dentro del nivel, también habrá que añadir ciertos elementos y controlar algunos eventos en sus *scripts*:

- **Señales de victoria y derrota**: el *framework* emite por defecto una señal de derrota (de victoria si se indica el modo *survival* en el minijuego) al acabar el tiempo especificado. Sin embargo, el usuario tendrá que programar los eventos que determinan la victoria/derrota por cualquier otra circunstancia (alcanzar un objetivo, cometer un error...). Esto se consigue incluyendo en el *script* del minijuego una señal `signal win`, la cual se activa a través de la función `emit_signal("win", <true(victoria)/false(derrota)>)`.
- **Detener ejecución**: con relación al aspecto anterior, es necesario paralizar la ejecución del minijuego tras finalizar, puesto que mantenerlo activo podría causar errores que cambien el resultado del mismo. Una manera sencilla de conseguirlo es deteniendo todos los procesos del nodo actual (`$"`) y sus hijos.

```
func _pause_recursively(node: Node) -> void:
    node.set_process(false)
    node.set_physics_process(false)
    node.set_process_input(false)
    node.set_process_unhandled_input(false)
    node.set_process_unhandled_key_input(false)
    for child in node.get_children():
        if child is Node:
            _pause_recursively(child)
```

- **Reproducir música y sonidos**: en caso de querer introducir música y sonidos en un minijuego, se deberá emplear una instancia existente del gestor de música del *framework*. Obtenemos una referencia a esta instancia mediante `get_tree().get_root().get_node("MusicManager")` y utilizamos sus funciones `play_music(<load("musica.mp3")>)` y

```
play_sound(<load("sonido.mp3")>).
```

- **Aumentar dificultad:** también se ofrece la posibilidad de obtener la puntuación del nivel para modificar el minijuego y aumentar la dificultad dependiendo de lo avanzado que esté el nivel. Para ello se debe incluir la siguiente función *setter* con la que el gestor del nivel comunica al minijuego la puntuación.

```
var score
func set_game_info(levelScore):
    score = levelScore
```

5. EXTRA: Puntuaciones codificadas

Con tal de evitar modificaciones indebidas en las puntuaciones guardadas, el archivo de guardado se almacena cifrado mediante AES. Esto significa que su contenido es ilegible sin la contraseña, por lo que no es posible editarlo directamente. Para resetear el progreso guardado basta con borrar el archivo `data.json` de la carpeta de datos del juego, que en Windows se encuentra en:

```
C:\Users\<usuario>\AppData\Roaming\Godot\app_userdata\<proyecto>\
```